Informe práctico

30 consultas SQL para resolver con Django ORM

Asignatura: Programación Backend con Django
Actividad: Interpretación de SQL y construcción de consultas ORM
Docente: Benjamín Mora Torres

1. Introducción y objetivo

Esta actividad fortalece la comprensión de las consultas a bases de datos relacionales mediante la traducción de 30 sentencias SQL al ORM de Django. Cada ejercicio presenta una pregunta y una consulta SQL de referencia. El estudiante deberá escribir, ejecutar y explicar la solución ORM correspondiente.

2. Estructura de la base de datos

- Categoria: id, nombre, descripcion, color, icono, fecha_creacion.
- Persona: id, rut, nombre_completo, email, telefono, ciudad, fecha_registro.
- Evento: id, titulo, descripcion, categoria_id, valor, es_gratuito, fecha_inicio, fecha_fin, ubicacion, es_online, capacidad_maxima, organizador, estado, fecha_creacion.
- Asistencia: id, evento_id, persona_id, fecha_registro, asistio, precio_pagado, codigo_ticket.

Relaciones: Categoria 1:N Evento; Evento 1:N Asistencia; Persona 1:N Asistencia. Evento y Persona tienen una relación N:M mediante Asistencia. La combinación (evento, persona) es única.

3. Instrucciones

Resuelva los 30 ejercicios utilizando Django ORM, sin raw() ni ejecución directa de SQL. Para cada ejercicio incluya el código ORM, evidencia del resultado y una breve explicación de la operación. Los nombres SQL app_* son referenciales: reemplace app por el nombre real de su aplicación Django.

4. Consultas básicas

Ejercicio 1. Listar todas las categorías registradas.

```
SELECT * FROM app_categoria;
```

Solución ORM: _____

Ejercicio 2. Mostrar nombre, RUT y correo electrónico de todas las personas.

```
SELECT nombre_completo, rut, email
FROM app_persona;
```

Solución ORM: _____

Ejercicio 3. Listar todos los eventos programados.

```
SELECT * FROM app_evento
WHERE estado = 'programado';
```

Solución ORM: _____

Ejercicio 4. Obtener todos los eventos gratuitos.

```
SELECT * FROM app_evento
WHERE es_gratuito = TRUE OR valor = 0;
```

Solución ORM: _____

Ejercicio 5. Mostrar eventos con valor superior a $10.000.

```
SELECT titulo, valor
FROM app_evento
WHERE valor > 10000;
```

Solución ORM: _____

Ejercicio 6. Listar personas ordenadas alfabéticamente.

```
SELECT * FROM app_persona
ORDER BY nombre_completo ASC;
```

Solución ORM: _____

4. Filtros y condiciones

Ejercicio 7. Buscar eventos cuyo título contenga Python sin distinguir mayúsculas.

```
SELECT * FROM app_evento
WHERE LOWER(titulo) LIKE '%python%';
```

Solución ORM: _____

Ejercicio 8. Mostrar eventos online que estén programados.

```
SELECT * FROM app_evento
WHERE es_online = TRUE AND estado = 'programado';
```

Solución ORM: _____

Ejercicio 9. Obtener eventos con valor entre $5.000 y $20.000.

```
SELECT titulo, valor FROM app_evento
WHERE valor BETWEEN 5000 AND 20000;
```

Solución ORM: _____

Ejercicio 10. Mostrar eventos que no estén cancelados.

```
SELECT * FROM app_evento
WHERE estado <> 'cancelado';
```

Solución ORM: _____

Ejercicio 11. Buscar personas de Santiago o Valparaíso.

```
SELECT * FROM app_persona
WHERE ciudad IN ('Santiago', 'Valparaíso');
```

Solución ORM: _____

Ejercicio 12. Mostrar los cinco eventos más recientes por fecha de inicio.

```
SELECT * FROM app_evento
ORDER BY fecha_inicio DESC LIMIT 5;
```

Solución ORM: _____

4. Relaciones entre modelos

Ejercicio 13. Mostrar título de cada evento y nombre de su categoría.

```
SELECT e.titulo, c.nombre AS categoria
FROM app_evento e INNER JOIN app_categoria c
ON e.categoria_id = c.id;
```

Solución ORM: _____

Ejercicio 14. Listar personas inscritas en cada evento.

```
SELECT e.titulo, p.nombre_completo
FROM app_asistencia a
INNER JOIN app_evento e ON a.evento_id = e.id
INNER JOIN app_persona p ON a.persona_id = p.id;
```

Solución ORM: _____

Ejercicio 15. Mostrar participantes que asistieron efectivamente.

```
SELECT e.titulo, p.nombre_completo
FROM app_asistencia a
INNER JOIN app_evento e ON a.evento_id = e.id
INNER JOIN app_persona p ON a.persona_id = p.id
WHERE a.asistio = TRUE;
```

Solución ORM: _____

Ejercicio 16. Obtener eventos de la categoría Tecnología.

```
SELECT e.* FROM app_evento e
INNER JOIN app_categoria c ON e.categoria_id = c.id
WHERE c.nombre = 'Tecnología';
```

Solución ORM: _____

Ejercicio 17. Mostrar personas con al menos una inscripción.

```
SELECT DISTINCT p.* FROM app_persona p
INNER JOIN app_asistencia a ON p.id = a.persona_id;
```

Solución ORM: _____

Ejercicio 18. Mostrar personas sin ninguna inscripción.

```
SELECT p.* FROM app_persona p
WHERE NOT EXISTS (SELECT 1 FROM app_asistencia a
WHERE a.persona_id = p.id);
```

Solución ORM: _____

4. Agrupaciones y funciones

Ejercicio 19. Contar todos los eventos.

```
SELECT COUNT(*) AS total_eventos FROM app_evento;
```

Solución ORM: _____

Ejercicio 20. Calcular valor promedio de los eventos pagados.

```
SELECT AVG(valor) AS promedio FROM app_evento
WHERE valor > 0;
```

Solución ORM: _____

Ejercicio 21. Contar eventos por categoría, incluyendo categorías vacías.

```
SELECT c.nombre, COUNT(e.id) AS total
FROM app_categoria c LEFT JOIN app_evento e
ON e.categoria_id = c.id
GROUP BY c.id, c.nombre;
```

Solución ORM: _____

Ejercicio 22. Contar inscritos por evento, incluyendo eventos sin inscripciones.

```
SELECT e.titulo, COUNT(a.id) AS inscritos
FROM app_evento e LEFT JOIN app_asistencia a
ON a.evento_id = e.id
GROUP BY e.id, e.titulo;
```

Solución ORM: _____

Ejercicio 23. Calcular recaudación por evento, incluyendo los que no tienen pagos.

```
SELECT e.titulo, COALESCE(SUM(a.precio_pagado), 0) AS recaudacion
FROM app_evento e LEFT JOIN app_asistencia a
ON a.evento_id = e.id
GROUP BY e.id, e.titulo;
```

Solución ORM: _____

Ejercicio 24. Mostrar categorías con más de tres eventos.

```
SELECT c.nombre, COUNT(e.id) AS total
FROM app_categoria c INNER JOIN app_evento e
ON e.categoria_id = c.id
GROUP BY c.id, c.nombre HAVING COUNT(e.id) > 3;
```

Solución ORM: _____

4. Consultas avanzadas

Ejercicio 25. Mostrar tres eventos con mayor cantidad de inscripciones.

```
SELECT e.titulo, COUNT(a.id) AS inscritos
FROM app_evento e LEFT JOIN app_asistencia a
ON a.evento_id = e.id
GROUP BY e.id, e.titulo
ORDER BY inscritos DESC LIMIT 3;
```

Solución ORM: _____

Ejercicio 26. Obtener personas inscritas en más de dos eventos.

```
SELECT p.nombre_completo, COUNT(a.id) AS total
FROM app_persona p INNER JOIN app_asistencia a
ON a.persona_id = p.id
GROUP BY p.id, p.nombre_completo HAVING COUNT(a.id) > 2;
```

Solución ORM: _____

Ejercicio 27. Mostrar eventos con inscripciones iguales o superiores a su capacidad.

```
SELECT e.titulo, e.capacidad_maxima, COUNT(a.id) AS inscritos
FROM app_evento e LEFT JOIN app_asistencia a
ON a.evento_id = e.id
GROUP BY e.id, e.titulo, e.capacidad_maxima
HAVING COUNT(a.id) >= e.capacidad_maxima;
```

Solución ORM: _____

Ejercicio 28. Obtener personas que asistieron efectivamente a eventos gratuitos.

```
SELECT DISTINCT p.* FROM app_persona p
INNER JOIN app_asistencia a ON a.persona_id = p.id
INNER JOIN app_evento e ON a.evento_id = e.id
WHERE a.asistio = TRUE
AND (e.es_gratuito = TRUE OR e.valor = 0);
```

Solución ORM: _____

Ejercicio 29. Mostrar eventos con valor superior al promedio de eventos pagados.

```
SELECT titulo, valor FROM app_evento
WHERE valor > (SELECT AVG(valor) FROM app_evento WHERE valor > 0);
```

Solución ORM: _____

Ejercicio 30. Generar reporte de categoría, inscritos, asistentes efectivos y recaudación.

```
SELECT e.titulo, c.nombre AS categoria,
COUNT(a.id) AS inscritos,
SUM(CASE WHEN a.asistio = TRUE THEN 1 ELSE 0 END) AS asistentes_efectivos,
COALESCE(SUM(a.precio_pagado), 0) AS recaudacion
FROM app_evento e
INNER JOIN app_categoria c ON e.categoria_id = c.id
LEFT JOIN app_asistencia a ON a.evento_id = e.id
GROUP BY e.id, e.titulo, c.id, c.nombre
ORDER BY recaudacion DESC;
```

Solución ORM: _____

5. Ejemplo de resolución

Ejercicio 3: Listar los eventos programados.

```
from .models import Evento

eventos = Evento.objects.filter(estado="programado")
for evento in eventos:
    print(evento.titulo, evento.estado)
```

Este ejemplo es orientativo. Los otros ejercicios deben ser resueltos por el estudiante.

6. Entregables

1. Portada con identificación.
2. Las 30 consultas SQL y sus equivalentes ORM.
3. Evidencias de ejecución y resultados.
4. Explicación de las consultas avanzadas.
5. Conclusión comparando SQL y Django ORM.

7. Criterios de evaluación sugeridos

Criterio	Ponderación
Consultas básicas (1–6)	15%
Filtros y condiciones (7–12)	15%
Relaciones (13–18)	20%
Agrupaciones (19–24)	20%
Consultas avanzadas (25–30)	25%
Presentación y evidencias	5%
TOTAL	100%

8. Consideraciones técnicas

Para trabajar con el ORM puede utilizar python manage.py shell. En Asistencia, la inscripción y la asistencia efectiva son conceptos distintos; la recaudación se obtiene de precio_pagado y no del valor publicado del evento. Las consultas SQL son de referencia y pueden requerir adaptaciones según el motor de base de datos.